

Dynamic Behavior Profiler: Runtime Behavioral Monitoring and Anomaly Detection for Android Applications

Abdeljalil Fajri^a, Mohammed Ait Ourajli^a, Yasser Chettour^a, Zakaria Aouianti^a

^a*École Nationale des Sciences Appliquées de Marrakech (ENSAM), Cadi Ayyad University, Marrakech, Morocco*

Abstract

Dynamic Behavior Profiler is an open-source, end-to-end behavioral surveillance system for Android applications. The system collects runtime signals via the Android Debug Bridge (ADB), applies machine learning-based anomaly detection using weighted z-score deviations, and exposes results through a REST API coupled with an interactive web dashboard. Detected anomalies are automatically classified according to the MITRE ATT&CK for Mobile framework, and root-cause hypotheses are generated on demand by a large language model. Experiments conducted on a dedicated Android test application demonstrate the system’s ability to identify abnormal behaviors — including infinite retry loops, network request spam, crash storms, and timeout patterns — with a detection latency under three seconds and zero false positives in the normal operating scenario.

Keywords: Android, Anomaly Detection, Mobile Security, ADB, MITRE ATT&CK, Machine Learning, Runtime Profiling, Flask

Required Metadata

1. Motivation and Significance

The security of Android mobile applications constitutes a major challenge in the current context of digital transformation. Android applications can exhibit abnormal runtime behaviors — whether resulting from defective design, malicious attacks, or software errors — and these behaviors are often difficult to detect manually [1].

Traditional static code analysis approaches fail to capture dynamic behaviors that only emerge at runtime, such as infinite retry loops, network saturation through repeated requests, or cyclic crash patterns. Dynamic analysis

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v1.0.0
C2	Permanent link to code/repository used for this code version	https://github.com/Abdeljalil-Fajri/AndroidBehaviorAnalysis.git
C3	Permanent link to Reproducible Capsule	https://github.com/Abdeljalil-Fajri/AndroidBehaviorAnalysis/blob/main/README.md
C4	Legal Code License	MIT License
C5	Code versioning system used	Git
C6	Software code languages, tools, and services used	Java, Android SDK API 37, Python 3.x, Flask, HTML5, JavaScript, CSS3, ADB
C7	Compilation requirements, operating environments & dependencies	Android Studio, Python 3.x, ADB (Android SDK Platform Tools), Flask 3.1, Flask-CORS, OkHttp, Windows/macOS/Linux, Android Emulator (Pixel 6 Pro AVD, API 37)
C8	Link to developer documentation/manual	https://github.com/Abdeljalil-Fajri/AndroidBehaviorAnalysis/blob/main/README.md
C9	Support email for questions	A.fajri8120@uca.ac.ma

Table 1: Code metadata (mandatory)

systems such as DroidBox [2] propose sandboxed analysis of Android applications but are limited to forensic analysis and do not provide real-time surveillance. CopperDroid [3] reconstructs application behavior via system call analysis but requires a complex virtualized infrastructure. Log-based anomaly detection systems such as DeepLog [4] apply deep learning to identify abnormal patterns in application logs, but are not tailored to the specific constraints of the Android runtime environment.

Furthermore, the MITRE ATT&CK for Mobile framework [5] provides a taxonomy of attack tactics and techniques on mobile platforms. To our knowledge, no existing tool integrates this classification directly into a real-time Android behavioral profiler accessible to development teams without

requiring complex infrastructure.

There is therefore a genuine need for tools capable of profiling the runtime behavior of Android applications in an automated and non-intrusive manner. Dynamic Behavior Profiler addresses this gap by offering an end-to-end behavioral monitoring system that integrates (1) signal collection via ADB, (2) machine learning-based anomaly detection, (3) MITRE ATT&CK classification, and (4) AI-powered root-cause analysis — all in a single, easily deployable open-source platform.

The system is designed for Android developers and security teams who need to observe behavioral anomalies in real time without modifying the application under analysis. It is particularly suited to quality assurance pipelines, penetration testing workflows, and Security Operations Center (SOC) environments.

2. Software Description

This section describes the architecture, internal processes, and functionalities of Dynamic Behavior Profiler. The system enables security analysts and developers to monitor the runtime behavior of Android applications, detect anomalies automatically, and obtain AI-generated diagnostic insights — all without instrumentation or modification of the analyzed application.

2.1. Software Architecture

The system architecture is designed around a decoupled, three-tier model that ensures high cohesion and scalability, as illustrated in Figure 1. This layered approach cleanly separates the generation of telemetry from its analysis and presentation.

1. **Generation Layer (Data Source):** At the foundation, the Android test application (`BehaviorTestApp`) acts as the controlled environment. It generates both normal operational traffic and simulated threat patterns via a graphical interface running on an Android emulator. The telemetry generated is entirely native, relying on standard Android SDK mechanisms without requiring root access or code injection.
2. **Analysis Layer (Core Processing):** This tier forms the intelligence of the system. A Python-based profiler continuously polls behavioral signals via the Android Debug Bridge (ADB). It processes these signals through a statistical anomaly detection engine utilizing z-score derivations. Furthermore, this layer orchestrates interactions with a local Large Language Model (Ollama/phi3) to generate root-cause hypotheses, ensuring sensitive application data never leaves the host machine.

The processed intelligence is then exposed through a lightweight REST API built with Flask [7].

3. **Visualization Layer (Presentation):** A responsive, single-page web dashboard consumes the REST API. It serves as a Security Operations Center (SOC) interface, polling the backend every second to render live metrics, MITRE-classified alerts, and actionable AI insights.

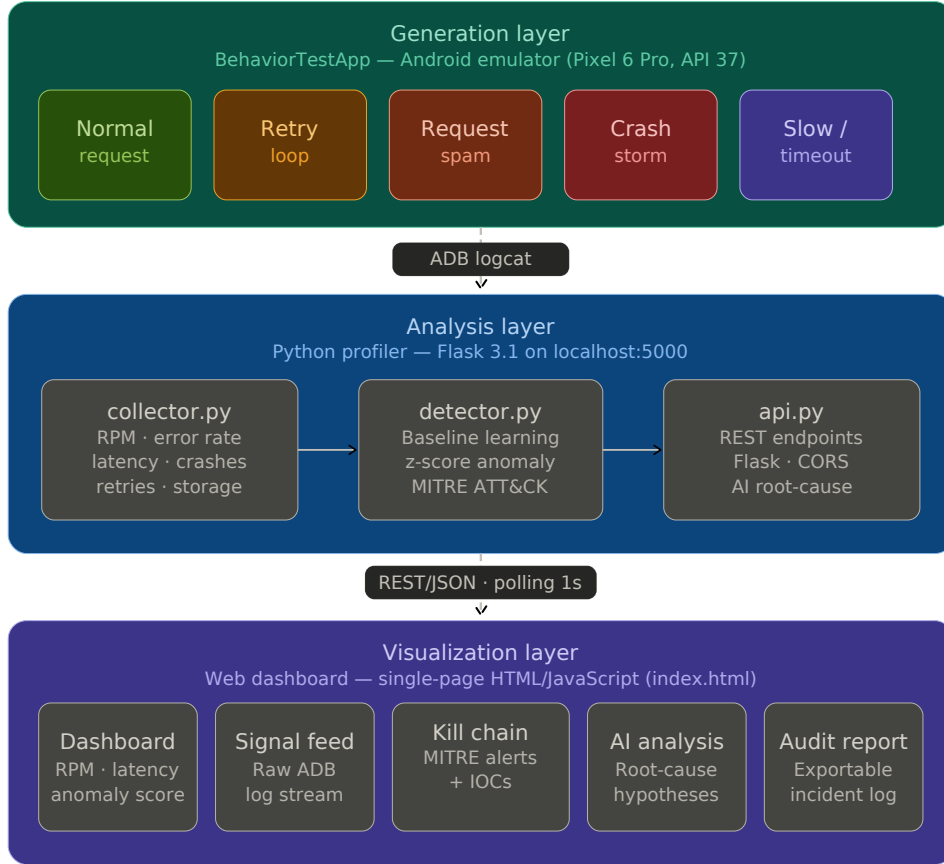


Figure 1: Global architecture of Dynamic Behavior Profiler, highlighting the decoupled interaction between the Android environment, the Python analysis backend, and the SOC web dashboard.

The technology stack used across the three layers is summarized in Table 2.

BehaviorTestApp — Android Test Application

The Android test application, developed with Android SDK API 37, implements five behavior modes accessible via a graphical interface:

Component	Technology
Android Application Emulator	Java, Android SDK API 37, OkHttp [6]
Signal Collection	Android Studio, Pixel 6 Pro AVD
REST API	Python 3.x, ADB (subprocess)
Anomaly Detection	Flask 3.1, Flask-CORS
Frontend	Python (statistics, z-score)
Communication	HTML5, JavaScript, CSS3
	REST/JSON, Server-Sent Events

Table 2: Technology stack of the Dynamic Behavior Profiler.

- **Normal Request:** Generates regularly spaced HTTP requests to `httpbin.org`, simulating normal application behavior.
- **Retry Loop:** Implements retry logic without exponential back-off, leading to an infinite retry loop on network errors.
- **Request Spam:** Generates a high volume of simultaneous requests using an `ExecutorService` with a pool of 8 threads.
- **Crash Storm:** Triggers repeated exceptions at regular intervals, simulating a cyclic crash pattern.
- **Slow / Timeout:** Uses short connection timeouts combined with slow endpoints to generate timeout patterns.

Network communication is handled by the OkHttp library [6] with connection and read timeouts configured at 5 seconds.

Signal Collection Engine — `collector.py`

The `collector.py` module is responsible for collecting behavioral signals from the Android emulator via ADB. It maintains a sliding history of signals and computes the following aggregated metrics:

- Requests per minute (RPM)
- Network error rate
- Average request latency
- Crash and retry frequency (per second)
- Storage file access count

Collection is performed via the `adb logcat` command, filtered on the tags `BehaviorApp`, `BehaviorApp.Network`, `BehaviorApp.Storage`, and `BehaviorApp.Crash`, providing fine-grained observation of application events.

Detection Engine — `detector.py`

The `detector.py` module implements anomaly detection in two phases.

Baseline phase: During the first minutes of execution, the system learns the normal behavior of the application by computing reference statistics (mean, standard deviation) for each collected metric.

Detection phase: Once the baseline is established, each new signal snapshot is compared against the reference. The anomaly score is calculated as a weighted combination of z-score deviations across all metrics:

$$S_{\text{anomaly}} = \sum_{i=1}^n w_i \cdot \max\left(0, \frac{x_i - \mu_i}{\sigma_i} - \theta\right) \quad (1)$$

where x_i is the current value of metric i , μ_i and σ_i are the baseline mean and standard deviation, w_i is the metric weight, and θ is the sensitivity threshold. The detection rules implemented and their MITRE ATT&CK correspondences are presented in Table 3.

Rule	MITRE ID	Severity
REQUEST_SPAM	T1499	HIGH
INFINITE_RETRY	T1071	MEDIUM
CRASH_LOOP	T1531	HIGH
TIMEOUT_PATTERN	T1040	MEDIUM
STORAGE_ANOMALY	T1005	LOW

Table 3: Detection rules and MITRE ATT&CK classification.

2.2. Process Workflow (BPMN)

To formalize the operational lifecycle of an incident within the system, the workflow is modeled using Business Process Model and Notation (BPMN). The end-to-end process is divided into three primary pools: Signal Collection, Anomaly Engine, and SOC Operations.

1. **Signal Ingestion & Baseline Learning:** The process initiates with the continuous ingestion of ADB logs. For the first three minutes, the system enters a learning phase to compute reference statistics (baseline).
2. **Real-Time Evaluation:** Once calibrated, continuous telemetry is evaluated against the baseline using Equation 1. If the anomaly score exceeds the dynamic threshold, an incident is triggered.
3. **Enrichment & AI Analysis:** Triggered incidents are instantly mapped to corresponding MITRE ATT&CK tactics. Simultaneously, the system queries the local LLM to generate contextualized remediation steps.

4. **Triage Workflow:** The incident is surfaced on the SOC dashboard where an operator must acknowledge it. The operator applies corrective measures and transitions the incident state to *Mitigated* or *False Positive*, closing the feedback loop.

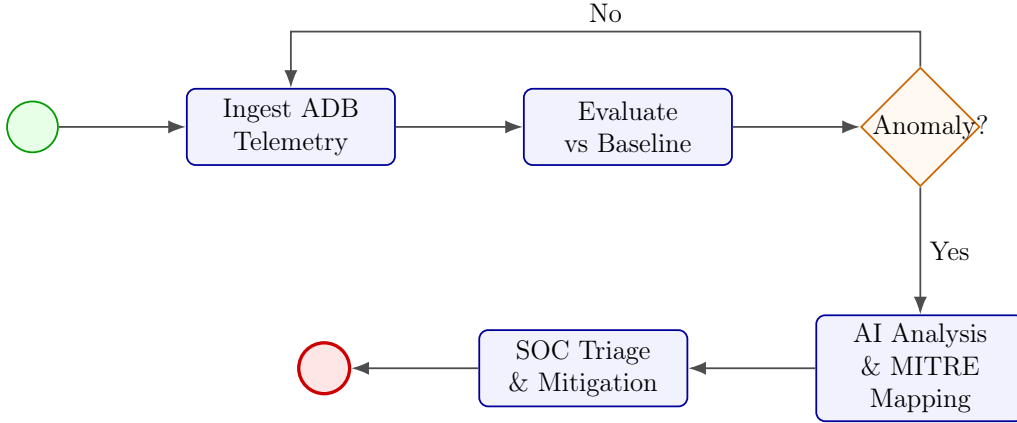


Figure 2: BPMN-inspired process diagram illustrating the incident lifecycle: from ADB signal ingestion and baseline comparison, to anomaly detection, AI enrichment, and manual SOC triage.

2.3. Software Functionalities

Dynamic Behavior Profiler offers the following key functionalities:

1. **Real-Time Signal Collection:** Continuous collection of behavioral metrics (RPM, error rate, latency, crashes, retries, storage accesses) from any running Android application via ADB, without requiring any modification to the application under analysis.
2. **Anomaly Detection:** Automated detection of behavioral anomalies using the z-score-based algorithm defined in Equation 1, with a configurable baseline learning phase and sensitivity threshold.
3. **MITRE ATT&CK Classification:** Each detected incident is automatically classified according to the MITRE ATT&CK for Mobile framework [5], providing a common language with security teams and enabling correlation with existing threat intelligence databases.
4. **REST API & Swagger Documentation:** A Flask-based API exposes all profiler data. To ensure interoperability and ease of integration for development teams, the entire API is documented following the OpenAPI Specification (OAS/Swagger).

5. **Interactive Web Dashboard:** A single-page HTML/JavaScript application querying the API every second, providing five views: *Dashboard* (live metrics and real-time charts), *Signal Feed* (raw log stream), *Kill Chain & Alerts* (MITRE-classified incidents with Indicators of Compromise), *AI Analysis* (root-cause hypotheses), and *Audit Report* (exportable incident report).
6. **AI-Powered Root-Cause Analysis:** On-demand generation of root-cause hypotheses by a large language model, contextualized with current metrics and active incidents.

Endpoint	Description
GET /api/status	Server status and baseline info
GET /api/signals/current	Real-time metrics snapshot
GET /api/signals/history	Historical signal data
GET /api/logs	Last 50 raw log entries
GET /api/incidents	List of detected anomalies
POST /api/analyze	Trigger AI root-cause analysis
POST /api/incidents/<id>/triage	Incident triage update

Table 4: Main REST API endpoints.

2.4. API Documentation (Swagger)

To facilitate seamless integration with third-party SIEM (Security Information and Event Management) platforms and CI/CD pipelines, the REST API is strictly documented using the OpenAPI 3.0 (Swagger) specification. The Swagger UI provides an interactive portal for developers to visualize endpoint schemas, understand required payloads, and test API calls directly from the browser.

Listing 1 presents a snippet of the OpenAPI specification defining the incident retrieval endpoint, ensuring standardized and predictable data exchange.

```
{
  "openapi": "3.0.0",
  "info": {
    "title": "Dynamic Behavior Profiler API",
    "version": "1.0.0"
  },
  "paths": {
    "/api/incidents": {
      "get": {
```

```

    "summary": "Retrieve detected anomalies",
    "responses": {
      "200": {
        "description": "A list of incidents mapped to MITRE ATT&CK",
        "content": {
          "application/json": {
            "schema": {
              "type": "array",
              "items": { "$ref": "#/components/schemas/Incident" }
            }
          }
        }
      }
    }
  }
}

```

Listing 1: OpenAPI/Swagger snippet defining the `/api/incidents` endpoint schema.

2.5. Sample Code Snippet

Listing 2 shows the JSON structure of a detected incident as returned by GET `/api/incidents`.

```

{
  "id": "uuid",
  "rule_name": "REQUEST_SPAM",
  "severity": "HIGH",
  "mitre_id": "T1499",
  "mitre_tactic": "Impact",
  "event_count": 47,
  "duration_sec": 12.3,
  "iocs": ["high_rpm", "error_rate_100"],
  "status": "open",
  "timestamp": "2026-05-17T15:10:00"
}

```

Listing 2: Incident data structure returned by the REST API.

3. Illustrative Examples

To demonstrate the practical use of Dynamic Behavior Profiler, we present four scenarios run on an Android Pixel 6 Pro emulator (API 37) on a Win-

dows PC with an Intel processor and 16 GB RAM. Each scenario was executed for 60 seconds following a 3-minute baseline learning period.

3.1. Scenario 1: Normal Request

The normal behavior produces a stable request rate and an anomaly score close to zero. The system generates no alerts, confirming the absence of false positives in this operating mode.

3.2. Scenario 2: Request Spam

During the request spam test, the dashboard recorded a peak of **4560 req/min** with a **100% error rate** and an **anomaly score of 40**. The REQUEST_SPAM incident (MITRE T1499, severity HIGH) was detected in less than **3 seconds** after triggering, as shown in Figure 3.

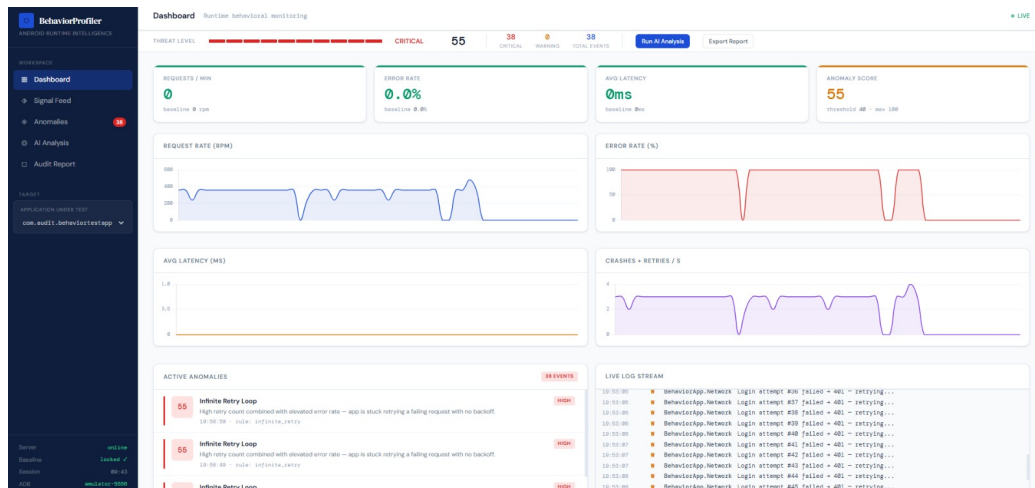


Figure 3: Dashboard during the Request Spam scenario: 4560 req/min, 100% error rate, anomaly score 40.

3.3. Scenario 3: Retry Loop

The infinite retry loop was detected via the elevation of the retries/second counter and the persistence of network errors. The INFINITE_RETRY incident (MITRE T1071) was raised with MEDIUM severity.

3.4. Scenario 4: Crash Storm

Repeated crashes caused a rise in the Crashes & Retries/s counter. The CRASH_LOOP incident (MITRE T1531) was detected and classified with HIGH severity.

3.5. AI Root-Cause Analysis

The AI Analysis feature generates root-cause hypotheses by contextualizing incidents with current metrics. For the Request Spam scenario, the system produced the following hypothesis:

“The abnormally high request volume (4560 RPM) combined with a 100% error rate suggests a retry loop without rate control. Probable cause: absence of client-side rate limiting or incorrect implementation of retry logic.”

Figure 4 shows the AI Analysis section of the dashboard with generated root-cause hypotheses.

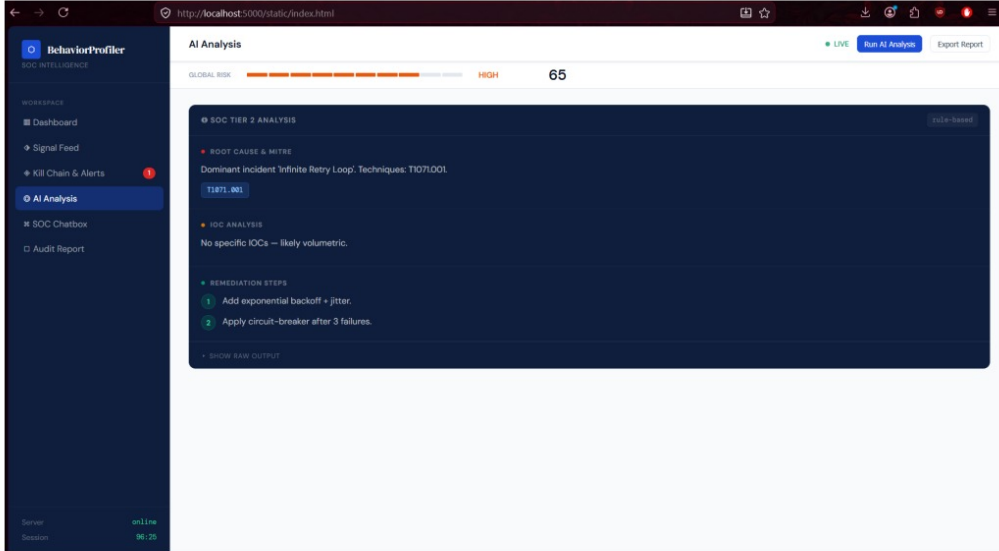


Figure 4: AI Analysis section of the dashboard showing root-cause hypotheses generated for the active incidents.

4. Impact

4.1. Comparison with Existing Tools

Dynamic Behavior Profiler distinguishes itself from existing Android analysis tools through its unique combination of real-time monitoring, automatic MITRE ATT&CK classification, and AI-powered diagnostics in a single deployable package. Table 5 positions it with respect to the closest related tools.

Feature	Ours	DroidBox	CopperDroid	DeepLog
Real-time monitoring	Yes	No	No	Partial
No app modification needed	Yes	No	No	Yes
MITRE ATT&CK mapping	Yes	No	No	No
AI root-cause analysis	Yes	No	No	No
Web dashboard	Yes	No	No	No
REST API	Yes	No	No	No
Open source	Yes	Yes	No	Yes
Android-native	Yes	Yes	Yes	No

Table 5: Comparison of Dynamic Behavior Profiler with related tools.

4.2. Key Contributions

Dynamic Behavior Profiler makes the following contributions to Android security and software quality assurance:

- **Non-intrusive surveillance:** ADB-based collection requires no modification of the application under analysis, enabling deployment on any debuggable Android device or emulator.
- **End-to-end integration:** The system is, to our knowledge, the first to combine real-time ADB-based collection, z-score anomaly detection, MITRE ATT&CK classification, and LLM-generated root-cause analysis in a single open-source tool.
- **SOC-ready classification:** MITRE ATT&CK mapping provides a common language with security teams and facilitates correlation with existing threat intelligence databases.
- **Low detection latency:** The system detects anomalies in under 3 seconds with a stable memory footprint of approximately 80 MB for the Python profiler process.
- **Extensibility:** The modular architecture (collector, detector, API, dashboard) allows independent updates to each component, and the detection rule engine can be extended with new MITRE-mapped rules without modifying the core framework.

4.3. Limitations and Future Directions

The system presents certain limitations. The dependency on ADB implies that the device must operate in debug mode, which is not always possible in production environments. Additionally, detection quality depends on the

duration of the baseline phase: a baseline that is too short may lead to false positives.

Future improvements envisaged include: (1) support for physical devices via Wi-Fi ADB, (2) integration of more sophisticated anomaly detection algorithms (Isolation Forest, LSTM autoencoders), (3) automatic export of reports in PDF format for SOC teams, and (4) extension of monitored signals to CPU and memory consumption patterns.

5. Quality Assurance

System performance was evaluated through controlled experiments on the scenarios described in Section 3. The key performance metrics are summarized in Table 6.

Metric	Result
Detection latency (Request Spam)	<3 seconds
Detection latency (Crash Storm)	<3 seconds
False positive rate (Normal mode)	0%
Collection latency (event to dashboard)	<1 second
Python profiler memory consumption	$\approx$ 80 MB (stable)
Baseline learning period	3 minutes

Table 6: System performance metrics.

The profiler introduces a collection latency of less than 1 second between the occurrence of an event and its display on the dashboard. Memory consumption of the Python process remains stable at approximately 80 MB throughout the duration of tests, confirming the system’s suitability for continuous monitoring sessions.

6. Conclusions

This paper presented Dynamic Behavior Profiler, an end-to-end open-source behavioral monitoring system for Android applications. The system integrates signal collection via ADB, z-score-based anomaly detection, MITRE ATT&CK classification, and automatic AI-generated root-cause analysis in a cohesive, easily deployable platform.

Experiments demonstrated the system’s ability to detect varied abnormal behaviors — request spam, retry loops, crash storms, and timeout patterns — with a detection latency under 3 seconds and no false positives in the normal operating scenario. The interactive web dashboard provides complete, real-time visibility into the behavioral health of analyzed applications.

This project constitutes a practical contribution to Android mobile application security, offering a tool that is open-source, extensible, and easily deployable for development and security teams alike. Future work will focus on extending support to physical devices, incorporating more sophisticated machine learning models, and broadening the set of monitored behavioral signals.

Acknowledgements

The authors thank Prof. Mohamed Lachgar for his supervision and guidance throughout the development of this project within the Android Mobile Application Programming and Security module at ENSAM Marrakech, Cadi Ayyad University.

References

- [1] N. Felt, M. Finifter, E. Chin, S. Hanna, and D. Wagner, “A Survey of Mobile Malware in the Wild,” in *Proc. ACM CCS Workshop on Security and Privacy in Smartphones and Mobile Devices*, 2011.
- [2] L. Piltz and T. Bläsing, “DroidBox: An Android Application Sandbox for Dynamic Analysis,” in *Proc. SICHERHEIT*, 2012.
- [3] K. Tam, S. Khan, A. Fattori, and L. Cavallaro, “CopperDroid: Automatic Reconstruction of Android Malware Behaviors,” in *Proc. NDSS Symposium*, 2015.
- [4] M. Du, F. Li, G. Zheng, and V. Srikumar, “DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning,” in *Proc. ACM CCS*, 2017, pp. 1285–1298.
- [5] MITRE Corporation, “MITRE ATT&CK for Mobile,” [Online]. Available: <https://attack.mitre.org/matrices/mobile/>, 2023.
- [6] Square Inc., “OkHttp: An HTTP Client for Android and Java,” [Online]. Available: <https://square.github.io/okhttp/>, 2023.
- [7] Pallets Projects, “Flask Web Framework,” [Online]. Available: <https://flask.palletsprojects.com/>, 2025.